

Agentic AI for Complex Task Execution: A Systematic Review of LLM-Based Agentic AI Architectures

1st Furqon Aryadana

*Department of Information Technology
Sepuluh Nopember Institute of
Technology
Surabaya, Indonesia
5027231024@student.its.ac.id*

2nd Kevin Anugerah Faza

*Department of Information Technology
Sepuluh Nopember Institute of
Technology
Surabaya, Indonesia
5027231027@student.its.ac.id*

3rd RM. Novian Malcolm Bayuputra

*Department of Information Technology
Sepuluh Nopember Institute of
Technology
Surabaya, Indonesia
5027231035@student.its.ac.id*

4th Rizka Wakhidatus Sholikhah

*Department of Information Technology
Sepuluh Nopember Institute of
Technology
Surabaya, Indonesia
wakhidatus@its.ac.id*

Abstract—Large Language Models (LLMs) have increasingly evolved from static text-generation systems into agentic systems capable of reasoning, planning, using tools, interacting with environments, and coordinating with other agents. However, existing studies on LLM-based agentic AI remain fragmented across different architectural patterns, task environments, evaluation settings, and reported limitations. This systematic literature review aims to identify, classify, and synthesize LLM-based agentic AI architectures for complex task execution. The review follows a structured systematic literature review protocol with PICOC-based scope formulation and transparent reporting principles. A final corpus of 33 studies was analyzed to examine architectural taxonomies, core agentic components, tasks, benchmarks, evaluation outcomes, and research gaps. The findings show that LLM-based agentic AI has evolved from reasoning-and-acting loops into modular systems that incorporate planning, reflection, memory, tool use, interface-based execution, embodied interaction, and multi-agent collaboration. The reviewed studies also indicate that agentic AI is increasingly evaluated in realistic environments, including web tasks, software engineering, API usage, mobile interaction, embodied settings, and general agent benchmarks. Despite these advances, major challenges remain in long-horizon planning, reliable tool execution, memory management, benchmark standardization, reproducibility, safety, and multi-agent coordination. This review provides a structured synthesis of current LLM-based agentic architectures and highlights future research directions toward more reliable, interpretable, and scalable agentic AI systems for complex task execution.

Keywords—Agentic AI, Large Language Models, LLM Agents, Complex Task Execution, Multi-Agent Systems

I. INTRODUCTION

A. From LLMs to Agentic AI

Large Language Models (LLMs) have transformed artificial intelligence by enabling systems to understand instructions, generate coherent responses, and perform complex reasoning over natural language inputs. However, many real-world tasks require more than a single generated answer. They often involve multi-step reasoning, tool use,

environmental interaction, self-correction, and context maintenance over time. This limitation has encouraged the development of LLM-based agents, where language models are embedded within architectures that allow them to plan, act, observe, and adapt during task execution.

Early agentic approaches show that LLM performance depends not only on the model itself, but also on the architecture that organizes reasoning, action, memory, and feedback. ReAct demonstrated that LLMs can interleave reasoning traces with external actions, while Reflexion showed that agents can improve future behavior through verbal feedback and self-reflection without updating model parameters [1], [2]. Agentic AI has also expanded beyond single-agent systems. Generative Agents combined memory, reflection, and planning for interactive behavior, while multi-agent debate showed that multiple LLM instances can improve reasoning through iterative critique [3], [4]. These studies mark a transition from LLMs as static text generators toward LLMs as components of autonomous or semi-autonomous task-execution systems.

B. LLM-Based Agentic Architectures for Complex Task Execution

Complex task execution refers to tasks that require multi-step reasoning, planning, tool invocation, memory retrieval, self-correction, or coordination across agents and environments. In this context, agentic architectures provide scaffolding around LLMs so they can decompose goals, select actions, evaluate intermediate results, and revise plans when necessary. Planning-oriented systems such as LATS and ADaPT show how agents can improve long-horizon reasoning through search, reflection, value estimation, or adaptive decomposition [5], [6].

Tool use and environment interaction are also central to LLM-based agentic systems. ToolLLM and HuggingGPT demonstrate how LLMs can select APIs, coordinate external models, and extend their capabilities beyond text generation [7], [8]. Meanwhile, SWE-agent, WorkArena, VisualWebArena, and AgentBench show that agentic AI is increasingly evaluated in realistic digital environments, including software repositories, web interfaces, enterprise

tasks, and interactive benchmarks [9]–[12]. Despite these advances, current agentic architectures still face challenges in long-horizon planning, tool-use reliability, memory management, multi-agent coordination, benchmark comparability, and reproducibility.

C. Research Gap and Review Objectives

The literature on LLM-based agentic AI remains diverse but fragmented. Some studies focus on reasoning-and-acting loops, while others emphasize planning, reflection, tool use, software engineering, web automation, memory, or multi-agent collaboration. Although these works provide important architectural contributions, there is still limited synthesis that specifically compares LLM-based agentic architectures for complex task execution. In particular, the field lacks a consolidated understanding of how different architectures support reasoning, planning, memory, tool use, self-correction, and collaboration across tasks and benchmarks.

This systematic literature review aims to identify, classify, and synthesize LLM-based agentic AI architectures used for complex task execution. The review focuses on architectural patterns, core agentic components, task environments, benchmark settings, reported outcomes, and limitations. This review is guided by the following research questions:

- **RQ1:** What types of LLM-based agentic AI architectures have been proposed for complex task execution?
- **RQ2:** What core architectural components and capabilities are commonly incorporated in LLM-based agentic AI systems for complex task execution?
- **RQ3:** In what tasks, domains, datasets, benchmark environments, and evaluation settings are LLM-based agentic AI architectures studied, and what outcomes are reported?
- **RQ4:** What limitations, open challenges, and research gaps are reported in LLM-based agentic AI systems for complex task execution?

The remainder of this paper is organized as follows. Section II presents the related work and conceptual foundations of LLM-based agentic AI, including benchmarks, task environments, and evaluation metrics. Section III describes the review methodology based on Kitchenham, PICOC, and PRISMA. Section IV presents the results according to the four research questions. Section V discusses the main findings and their implications. Section VI provides recommendations and future research directions. Finally, Section VII concludes the paper.

II. METHODOLOGY

A. Define PICOC and RQs

This systematic literature review was conducted using a structured protocol organized into three main stages: planning, conducting, and reporting. The planning stage defined the review objectives, scope, search strategy, inclusion and exclusion criteria, and quality assessment procedure. The conducting stage involved identifying candidate studies, applying selection criteria, assessing study quality, and organizing the final reference corpus. The reporting stage synthesized the selected literature and presented the results in the form of architectural taxonomy,

component analysis, benchmark mapping, and limitation synthesis.

PICOC was used to define the review scope and align the research questions with the study selection process. The Population consisted of LLM-based agents, autonomous agents, agentic AI systems, and LLM-based multi-agent systems. The Intervention focused on LLM-based agentic architectures for complex task execution, including ReAct-style agents, planning agents, reflection agents, tool-augmented agents, memory-augmented agents, and multi-agent systems. The Comparison involved differences across agentic architectures or comparisons against non-agentic, single-agent, or baseline approaches when available. The Outcome included reasoning capability, planning capability, tool-use performance, task completion, robustness, error correction, benchmark performance, efficiency, and architectural limitations. The Context covered complex task execution, multi-step reasoning, long-horizon planning, task decomposition, tool invocation, interactive decision making, and workflow orchestration. PRISMA-style reporting principles were additionally adopted to ensure transparent documentation of the study identification, screening, eligibility, and inclusion process.

B. Search Strategy and Study Selection

The search strategy was designed to identify studies related to LLM-based agentic AI architectures for complex task execution. The initial search layer used ScienceDirect as the seed database with the keyword “Agentic AI”. The search was limited to Research Articles and Review Articles published between 2020 and 2026. This initial search produced 87,375 results and was used to establish the broad research coverage of the topic. Since the initial result set was intentionally broad, it was not directly used as the final evidence corpus. Instead, it served as a seed layer to demonstrate the growing scope of agentic AI research.

After the seed search, the candidate pool was refined using Elicit-based references, combined reference lists, non-ArXiv review papers, peer-reviewed primary studies, benchmark papers, framework papers, and architecture-focused studies related to LLM-based agents. Search terms and reference tracing emphasized concepts such as “LLM-based agent”, “language agent”, “agentic AI”, “tool-augmented agent”, “planning agent”, “memory-augmented agent”, “reflection agent”, “multi-agent system”, “web agent”, “software engineering agent”, and “complex task execution”. These terms were selected because they represent the core architectural dimensions of LLM-based agentic AI rather than general LLM usage.

The study selection process consisted of several stages. First, candidate studies were collected from the seed search layer and reference pool. Second, duplicate records and clearly irrelevant studies were removed. Third, a strict non-ArXiv policy was applied. A paper was not used as a final reference if its final venue or final record was arXiv, arXiv.org, an arXiv preprint, or an arXiv DOI. If a study had both an arXiv version and an official peer-reviewed version, only the official venue or publisher record was retained. Acceptable final records included sources such as ICLR, ACL Anthology, PMLR, NeurIPS proceedings, ACM Digital Library, IEEE Xplore, Springer, ScienceDirect, and other official publisher records.

Fourth, the remaining studies were assessed based on relevance to LLM-based agentic architecture and complex task execution. Studies were prioritized when they proposed an agent architecture, evaluated an agent system, introduced an agent benchmark, or analyzed architectural components such as reasoning, planning, memory, tool use, reflection, and multi-agent coordination. Finally, the resulting papers formed the fixed reference corpus for this review. The final corpus consisted of 33 papers. These papers were treated as the final selected references and were not subjected to further filtering during the writing stage of this review.

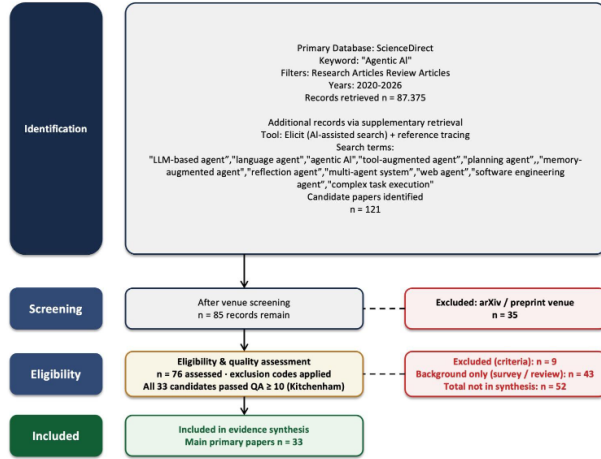


FIGURE I. PRISMA FLOW DIAGRAM OF THE SYSTEMATIC LITERATURE REVIEW PROCESS.

C. Inclusion, Exclusion, and Quality Assessment

The inclusion and exclusion criteria were defined to ensure that the selected studies were aligned with the scope of this review. Studies were included when they discussed or evaluated LLM-based agentic AI architectures for complex task execution, including multi-step reasoning, long-horizon planning, tool invocation, memory, reflection, or multi-agent coordination. Eligible studies had to be published between 2020 and 2026, written in English, available in full text, and published in a peer-reviewed journal, indexed conference proceeding, or official publisher venue. Studies were excluded when they did not use an LLM as the core reasoning component, focused only on prompting or fine-tuning without an agentic architecture, lacked substantial empirical or architectural contribution, duplicated another included study, had inaccessible full text, appeared only as an arXiv preprint, or did not meaningfully address complex task execution. Survey and review papers were not treated as primary evidence, although relevant non-ArXiv reviews were used as background references when appropriate.

Quality assessment was conducted to evaluate the methodological and architectural relevance of candidate studies. Seven criteria were considered: clarity of the proposed or evaluated agentic architecture, explicitness of the study objective, description of architectural components such as planning, memory, tool use, reflection, or coordination, use of benchmarks or clearly defined task settings, methodological detail for reproducibility, comparison with baselines or justification of effectiveness, and discussion of limitations or failure cases. Each criterion was scored from 0 to 2, where 0 indicated absent, 1 partially

addressed, and 2 fully addressed, with a maximum score of 14. Studies scoring 10–14 were included as primary evidence, studies scoring 8–9 were treated as borderline, and studies scoring 0–7 were excluded from the primary synthesis. This process ensured that the final corpus was not only relevant, but also sufficiently clear in architecture, evaluation, and reported limitations.

D. Data Synthesis and Bias Mitigation

The selected studies were synthesized using a qualitative thematic approach. Rather than treating each paper as an isolated contribution, the synthesis organized the literature based on architectural patterns, agent capabilities, task settings, evaluation methods, and reported limitations. This approach was appropriate because the included studies vary in terms of domain, benchmark, implementation, and evaluation metric. By grouping studies across common conceptual dimensions, the review could compare different agentic architectures while still preserving the distinctive contributions of each study.

The synthesis process was guided by the four research questions. For RQ1, the studies were analyzed to identify major types of LLM-based agentic AI architectures, such as reasoning-and-acting agents, planning agents, tool-augmented agents, memory-augmented agents, reflection agents, interface-based agents, embodied agents, and multi-agent systems. For RQ2, the analysis focused on core architectural components, including reasoning, planning, task decomposition, tool use, memory handling, self-reflection, error correction, and collaboration. For RQ3, the review mapped tasks, domains, datasets, benchmark environments, and evaluation outcomes. For RQ4, the synthesis identified recurring limitations, failure modes, open challenges, and future research gaps.

Several measures were applied to reduce bias and improve the reliability of the review. The scope was defined using PICOC before synthesis, while inclusion, exclusion, and quality assessment criteria were applied to reduce subjective selection. The strict non-ArXiv policy ensured that final references relied on official peer-reviewed or publisher records, and the findings were organized around predefined research questions to avoid selective grouping after analysis. Potential threats to validity include coverage bias due to the rapid development of agentic AI, publication bias from excluding preprint-only studies, and interpretation bias caused by overlapping architectural categories. These threats were mitigated through multi-source search, strict quality assessment, and analysis based on each study's dominant architectural contribution.

III. RESULTS

A. Characteristics of Included Studies

The final corpus consisted of 33 studies on LLM-based agentic AI architectures for complex task execution. The included studies were concentrated in recent years, reflecting the rapid emergence of language agents after the introduction of reasoning-and-acting, reflection, tool-use, and multi-agent paradigms. The studies varied in their dominant focus, including reasoning-and-acting agents, planning and search-based agents, tool-augmented agents, memory-augmented agents, web and software agents, embodied agents, multi-agent systems, and benchmark-oriented studies.

The corpus shows that LLM-based agentic AI is not a single architectural family. Instead, it is a collection of design patterns that combine LLMs with supporting mechanisms such as planning, tool invocation, memory retrieval, reflection, feedback loops, and multi-agent coordination. For example, ReAct and Reflexion focus on reasoning-action loops and self-reflective improvement [1], [2]. LATS, ADaPT, and related search-based methods focus on planning, decomposition, and decision optimization [5], [6], [14]. ToolLLM, HuggingGPT, AutoTools, and Meta-Agent-Workflow emphasize tool use and workflow execution [7], [8], [15], [16]. Meanwhile, WorkArena, VisualWebArena, SWE-agent, Mobile-Bench, AgentBench, and MMAU emphasize evaluation in realistic task environments and capability-oriented benchmarks [9]–[12], [25], [30].

Table I summarizes the dominant focus areas observed in the included studies. The categories are not mutually exclusive in terms of architectural components because many systems combine planning, tool use, memory, reflection, and evaluation. However, each study was grouped based on its primary contribution to the literature.

TABLE I. DOMINANT FOCUS AREAS OF INCLUDED STUDIES

Dominant Focus Area	Main Emphasis	Representative Studies
Reasoning and Acting	Interleaving reasoning with action, improving behavior through critique or self-reflection	ReAct [1], Reflexion [2], Multiagent Debate [4], Table-Critic [19]
Planning, search, and decomposition	Task decomposition, tree search, world modeling, adaptive planning, and planning improvement	LATS [5], ADaPT [6], LLM-MCTS [13], tree-search analysis [14], AgentGen [22], WebDreamer [29], Tree Search for LM Agents [31]
Tool use and workflow orchestration	API use, external model orchestration, tool documentation parsing, and reusable workflow construction	ToolLLM [7], HuggingGPT [8], AutoTools [15], Meta-Agent-Workflow [16], LLMind [32]
Memory and long-term context	Memory retrieval, memory augmentation, behavioral continuity, and experience reuse	Generative Agents [3], MemInsight [17], ACAN memory retrieval [18]
Web, software, and mobile agents	Agent interaction with repositories, web interfaces, enterprise software, and mobile apps	SWE-agent [9], WorkArena [10], VisualWebArena [11], Mobile-Bench [30]
Benchmark and capability evaluation	General agent evaluation, capability breakdown, collaboration evaluation, and task completion assessment	AgentBench [12], MMAU [25], MultiAgentBench [28]

Multi-agent and embodied collaboration	Role specialization, debate, heterogeneous agents, theory of mind, and embodied cooperation	AutoAgents [20], Internet of Agents [23], CoELA [24], Hypothetical Minds [33]
Domain-specific agentic systems	Agentic reasoning in applied technical domains	RCA agents [26], LLMind [32]

Overall, the included studies indicate a clear shift from prompt-only LLM use toward modular agent architectures. Instead of relying only on the language model’s internal knowledge, recent systems increasingly combine the LLM with external tools, task environments, feedback loops, memory modules, symbolic planners, tree search, or other agents. This shift is central to understanding agentic AI as an architectural problem rather than only a model capability problem. These focus areas are analyzed in detail in the following subsections, organized according to the four research questions.

B. RQ1: Taxonomy of LLM-Based Agentic AI Architectures

The first research question examines the types of LLM-based agentic AI architectures proposed for complex task execution. Based on the reviewed studies, this review identifies eight major architectural types: reasoning-and-acting agents, reflection agents, planning and search-based agents, tool-augmented agents, memory-augmented agents, interface-based agents, embodied agents, and multi-agent systems. These categories are not fully separate, because many agentic systems combine multiple capabilities such as planning, tool use, memory, reflection, and collaboration within a single architecture.

Reasoning-and-acting agents form the foundation of LLM-based agentic AI by connecting internal reasoning with external actions, as demonstrated by ReAct [1]. Reflection agents extend this pattern by using feedback or verbal self-reflection to improve future decisions without updating model parameters, as shown in Reflexion and Table-Critic [2], [19]. Planning and search-based agents, including LATS, ADaPT, LLM-MCTS, WebDreamer, and Tree Search for Language Model Agents, focus on task decomposition, search, simulation, and trajectory evaluation for long-horizon problem solving [5], [6], [13], [29], [31].

Other architectures extend agent capability through tools, memory, interfaces, embodiment, and collaboration. Tool-augmented agents such as ToolLLM, HuggingGPT, AutoTools, and Meta-Agent-Workflow enable LLMs to call APIs, coordinate external models, and construct reusable workflows [7], [8], [15], [16]. Memory-augmented agents such as Generative Agents, MemInsight, and ACAN-based retrieval support long-term context and experience reuse [3], [17], [18]. Interface-based and embodied agents operate in software repositories, web interfaces, mobile applications, or physical environments, while multi-agent systems distribute reasoning and execution across specialized agents [9]–[11], [20], [23], [24], [28], [30], [33]. Overall, the taxonomy shows that agentic AI has shifted from simple prompting toward modular architectures that combine reasoning, planning, memory, tool use, environment interaction, and collaboration.

C. RQ2: Core Architectural Components and Capabilities

The second research question examines the core components and capabilities commonly incorporated in LLM-based agentic AI systems for complex task execution. Across the reviewed studies, the most frequent components are reasoning, planning, tool use, memory, reflection, environment interaction, and collaboration. These components do not usually appear in isolation; instead, most agentic systems combine several of them to support iterative decision making, action execution, feedback processing, and task completion.

Reasoning and planning are the foundation of most LLM-based agentic architectures. Reasoning enables agents to interpret goals, generate intermediate thoughts, select actions, and revise decisions based on observations, as shown in ReAct and multiagent debate [1], [4]. Planning extends this capability by helping agents decompose complex goals, explore possible action trajectories, simulate future states, or evaluate candidate solutions before execution. Studies such as LATS, ADaPT, LLM-MCTS, and Tree Search for Language Model Agents show that planning can improve long-horizon task execution, although its effectiveness depends on task complexity, search strategy, and the reliability of intermediate-state evaluation [5], [6], [13], [14], [31].

Tool use and environment interaction expand the practical capability of LLM-based agents beyond text generation. Tool-augmented systems such as ToolLLM, HuggingGPT, AutoTools, and Meta-Agent-Workflow allow agents to select APIs, generate valid parameters, coordinate external models, execute functions, and reuse workflows [7], [8], [15], [16]. Interface-based agents further show that performance depends not only on the LLM, but also on how the agent interacts with repositories, web pages, mobile applications, or other task environments [9]–[11], [30]. This indicates that agent capability is shaped by both internal reasoning and the external action space available to the system.

Memory, reflection, and collaboration support continuity, self-correction, and distributed problem solving. Memory mechanisms help agents retrieve prior experiences, maintain contextual information, and support long-term behavioral consistency, as shown in Generative Agents, MemInsight, and ACAN-based retrieval [3], [17], [18]. Reflection and self-correction allow agents to learn from failure through verbal feedback, critique, refinement, or execution feedback without necessarily updating model parameters [2], [5], [16], [19]. In multi-agent systems, collaboration distributes reasoning and execution across specialized agents through debate, role assignment, cooperative planning, or theory-of-mind scaffolding [4], [20], [23], [28], [33]. Overall, these components show that the effectiveness of agentic AI depends not only on the underlying LLM, but also on how the architecture organizes reasoning, planning, memory, tool use, feedback, environment interaction, and coordination.

D. RQ3: Tasks, Benchmarks, Datasets, and Evaluation Outcomes

The third research question examines how LLM-based agentic AI systems are evaluated across tasks, domains, benchmarks, datasets, and performance outcomes. As

summarized in Table II, the reviewed studies cover diverse evaluation settings, ranging from general interactive benchmarks and web-based tasks to software engineering, tool use, planning, embodied interaction, mobile and IoT execution, multi-agent collaboration, and domain-specific diagnosis. This variety shows that the evaluation of agentic AI has expanded beyond text-only tasks toward interactive, environment-based, and action-oriented assessment.

TABLE II. TASK ENVIRONMENTS AND EVALUATION OUTCOMES IN INCLUDED STUDIES

Task / Environment Type	Representative Studies	Common Evaluation Outcomes
General interactive agent benchmarks	AgentBench [12], MMAU [25]	Task success, capability score, planning, reasoning, tool execution
Web and enterprise task environments	WorkArena [10], VisualWebArena [11], WebDreamer [29]	Completion rate, navigation success, visual grounding, robustness
Software engineering tasks	SWE-agent [9]	Issue resolution, test passing, patch correctness
Tool-use and API tasks	ToolLLM [7], AutoTools [15], Meta-Agent-Workflow [16]	Tool-call accuracy, execution success, workflow correctness
Planning and search tasks	LATS [5], ADaPT [6], LLM-MCTS [13], Tree Search for LM Agents [31]	Success rate, planning quality, search efficiency
Embodied and physical task settings	AutoGPT+P [27], CoELA [24]	Task completion, cooperation success, action grounding
Mobile and IoT task execution	Mobile-Bench [30], LLMind [32]	UI operation success, API execution, device/module orchestration
Multi-agent collaboration	Multiagent Debate [4], AutoAgents [20], MultiAgentBench [28], Hypothetical Minds [33]	Collaboration quality, answer accuracy, coordination effectiveness
Domain-specific diagnosis	RCA agents [26]	Diagnostic accuracy, evidence collection, root-cause identification

The table shows that evaluation outcomes differ depending on the task environment. General benchmarks emphasize broad agent capabilities, web and software environments focus on task completion and interface interaction, tool-use studies measure execution correctness, while multi-agent and embodied settings require attention to coordination and action grounding. However, the evaluation landscape remains fragmented because benchmarks differ in task design, allowed actions, feedback mechanisms, and success criteria. As a result, direct comparison between agent architectures remains difficult, especially across studies that evaluate different environments such as web

navigation, API use, software engineering, embodied cooperation, or diagnostic reasoning.

E. RQ4: Limitations, Challenges, and Research Gaps

The fourth research question identifies the main limitations, challenges, and research gaps reported across the reviewed studies. The most frequent limitation is unreliable long-horizon planning, where agents still struggle with multi-step task execution, search depth, and reliable evaluation of intermediate states. Planning-based systems such as LATS, ADaPT, WebDreamer, and Tree Search for Language Model Agents show progress in decomposition, search, and trajectory evaluation, but their performance remains sensitive to task complexity, feedback quality, and the reliability of value functions or discriminators [5], [6], [14], [29], [31].

Tool-use reliability and memory management also remain major bottlenecks. Tool-augmented agents may select inappropriate tools, generate invalid parameters, misinterpret documentation, or fail to recover from execution errors, as observed in ToolLLM, AutoTools, and Meta-Agent-Workflow [7], [15], [16]. Meanwhile, memory-based agents such as Generative Agents, MemInsight, and ACAN-based retrieval show that memory can support long-term context and experience reuse, but agents may still retrieve irrelevant memories, miss important prior information, or accumulate noisy context over time [3], [17], [18].

Other unresolved challenges include multi-agent coordination, benchmark fragmentation, reproducibility, and safety. Multi-agent systems can support specialization and collaboration, but they also introduce communication cost, role ambiguity, and coordination failure [20], [23], [28]. Evaluation also remains fragmented because benchmarks differ in task design, action spaces, feedback mechanisms, and success criteria, making direct comparison between architectures difficult [10]–[12], [25]. In addition, results often depend on model versions, prompts, tools, interface design, and execution settings, which limits reproducibility [7], [9], [10]. Safety is another important gap because agentic systems can execute external actions, access APIs, modify files, or operate interfaces with possible unintended consequences. Overall, these limitations show that future research should prioritize robust planning, safer tool execution, scalable memory, standardized benchmarks, reproducible evaluation, and reliable multi-agent coordination.

IV. DISCUSSION

The findings show that LLM-based agentic AI should be understood as an architectural evolution rather than only an improvement in model capability. Early systems such as ReAct [1] showed that LLMs can become interactive problem solvers, while later work on planning [5], [6], tool use [7], [8], [15], [16], and multi-agent collaboration [4], [20] demonstrates a clear shift toward modular agent architectures. However, this modularity introduces new challenges: planning effectiveness depends on reliable state evaluation [13], [14], tool use remains vulnerable to execution errors, and memory retrieval is difficult to scale.

Agent performance is also strongly influenced by the environment in which the agent operates. SWE-agent, WorkArena, VisualWebArena, and AgentBench demonstrate

that realistic task execution involves interfaces, action spaces, feedback mechanisms, and environmental constraints [9]–[12]. This means that evaluation should measure not only final task success, but also how agents interact with tools, software, and dynamic environments. Overall, future research should focus on designing agent architectures that are not only capable, but also reliable, interpretable, reproducible, and safe for real-world complex task execution.

V. CONCLUSION AND FUTURE WORK

This systematic literature review examined LLM-based agentic AI architectures for complex task execution. The reviewed studies show that agentic AI has evolved from simple reasoning-action loops into modular systems that combine planning, memory, reflection, tool use, environment interaction, and multi-agent collaboration. The review identified eight major architectural categories and showed that agentic AI is evaluated across diverse environments, including web tasks, software engineering, API usage, mobile interaction, embodied settings, and general agent benchmarks. However, major challenges remain in long-horizon planning, reliable tool execution, memory management, benchmark standardization, reproducibility, safety, and multi-agent coordination.

Several directions for future research emerge from these findings. Planning mechanisms should be improved through approaches that combine decomposition, search, simulation, and verification while still controlling computational cost. Tool-augmented agents require stronger execution safeguards, including better tool selection, parameter validation, and error recovery. Memory systems need clearer evaluation standards for retrieval relevance, long-horizon usefulness, and privacy-aware handling. The field also needs more standardized and realistic benchmarks to support fair comparison across reasoning, planning, tool use, and collaboration capabilities. Finally, multi-agent systems require better communication protocols, role assignment strategies, and collaboration metrics. In summary, future progress will depend not only on stronger LLMs, but also on better agent architectures, more realistic benchmarks, safer execution mechanisms, and more rigorous evaluation standards.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in Proc. Int. Conf. Learning Representations (ICLR), 2023.
- [2] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in Proc. Conf. Neural Information Processing Systems (NeurIPS), 2023.
- [3] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in Proc. ACM Symp. User Interface Software and Technology (UIST), Oct. 2023.
- [4] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, “Improving factuality and reasoning in language models through multiagent debate,” in Proc. Int. Conf. Machine Learning (ICML), 2024.
- [5] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, “Language agent tree search unifies reasoning, acting, and planning in language models,” in Proc. Int. Conf. Machine Learning (ICML), 2024.
- [6] A. Prasad, A. Koller, M. Hartmann, P. Clark, A. Sabharwal, M. Bansal, and T. Khot, “ADaPT: As-needed decomposition and planning with language models,” in Findings of the Association for Computational Linguistics: NAACL, 2024.
- [7] Y. Qin et al., “ToolLLM: Facilitating large language models to master 16000+ real-world APIs,” in Proc. Int. Conf. Learning Representations (ICLR), 2024.

- [8] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face," in Proc. Conf. Neural Information Processing Systems (NeurIPS), 2023.
- [9] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in Proc. Conf. Neural Information Processing Systems (NeurIPS), 2024.
- [10] A. Drouin et al., "WorkArena: How capable are web agents at solving common knowledge work tasks?" in Proc. Int. Conf. Machine Learning (ICML), 2024.
- [11] J. Y. Koh, R. Lo, L. Jang, V. Duvvur, M. C. Lim, P.-Y. Huang, G. Neubig, S. Zhou, R. Salakhutdinov, and D. Fried, "VisualWebArena: Evaluating multimodal agents on realistic visually grounded web tasks," in Proc. Annu. Meeting Assoc. Computational Linguistics (ACL), 2024.
- [12] X. Liu et al., "AgentBench: Evaluating LLMs as agents," in Proc. Int. Conf. Learning Representations (ICLR), 2024.
- [13] Z. Zhao, W. S. Lee, and D. Hsu, "Large language models as commonsense knowledge for large-scale task planning," in Proc. Conf. Neural Information Processing Systems (NeurIPS), 2023.
- [14] Z. Chen, M. White, R. Mooney, A. Payani, Y. Su, and H. Sun, "When is tree search useful for LLM planning? It depends on the discriminator," in Proc. 62nd Annu. Meeting Assoc. Computational Linguistics (ACL), 2024, pp. 13659–13678.
- [15] Z. Shi, S. Gao, L. Yan, Y. Feng, X. Chen, Z. Chen, D. Yin, S. Verberne, and Z. Ren, "Tool learning in the wild: Empowering language models as automatic tool agents," in Proc. ACM Web Conf. (WWW), Apr. 2025.
- [16] X. Tan, B. Li, X. Qiu, C. Qu, W. Chu, Y. Xu, and Y. Qi, "Meta-Agent-Workflow: Streamlining tool usage in LLMs through workflow construction, retrieval, and refinement," in Companion Proc. ACM Web Conf. (WWW Companion), Apr. 2025.
- [17] R. Salama, J. Cai, M. Yuan, A. Currey, M. Sunkara, Y. Zhang, and Y. Benajiba, "MemInsight: Autonomous memory augmentation for LLM agents," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2025, pp. 33136–33152.
- [18] C. Hong and Q. He, "Enhancing memory retrieval in generative agents through LLM-trained cross attention networks," *Frontiers in Psychology*, vol. 16, Art. no. 1591618, 2025.
- [19] P. Yu, G. Chen, and J. Wang, "Table-Critic: A multi-agent framework for collaborative criticism and refinement in table reasoning," in Proc. 63rd Annu. Meeting Assoc. Computational Linguistics (ACL), 2025, pp. 17432–17451.
- [20] G. Chen, S. Dong, Y. Shu, G. Zhang, J. Sesay, B. Karlsson, J. Fu, and Y. Shi, "AutoAgents: A framework for automatic agent generation," in Proc. 33rd Int. Joint Conf. Artificial Intelligence (IJCAI), 2024.
- [21] T. Webb, S. S. Mondal, and I. Momennejad, "A brain-inspired agentic architecture to improve planning with LLMs," *Nature Communications*, vol. 16, Art. no. 8633, 2025.
- [22] M. Hu, P. Zhao, C. Xu, Q. Sun, J.-G. Lou, Q. Lin, P. Luo, and S. Rajmohan, "AgentGen: Enhancing planning abilities for large language model based agent via environment and task generation," in Proc. 31st ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD), Aug. 2025.
- [23] W. Chen et al., "Internet of agents: Weaving a web of heterogeneous agents for collaborative intelligence," in Proc. Int. Conf. Learning Representations (ICLR), 2025.
- [24] H. Zhang, W. Du, J. Shan, Q. Zhou, Y. Du, J. B. Tenenbaum, T. Shu, and C. Gan, "Building cooperative embodied agents modularly with large language models," in Proc. Int. Conf. Learning Representations (ICLR), 2024.
- [25] G. Yin et al., "MMAU: A holistic benchmark of agent capabilities across diverse domains," in Proc. North American Chapter of the Association for Computational Linguistics (NAACL), 2025, pp. 4752–4780.
- [26] D. Roy, X. Zhang, R. Bhawe, C. Bansal, P. Las-Casas, R. Fonseca, and S. Rajmohan, "Exploring LLM-based agents for root cause analysis," in Companion Proc. 32nd ACM Int. Conf. Foundations of Software Engineering (FSE Companion), 2024.
- [27] T. Birr, C. Pohl, A. Younes, and T. Asfour, "AutoGPT+P: Affordance-based task planning using large language models," in Proc. Robotics: Science and Systems (RSS), 2024.
- [28] X. Zhu et al., "MultiAgentBench: Evaluating the collaboration of LLM agents," in Proc. Annu. Meeting Assoc. Computational Linguistics (ACL), 2025.
- [29] Y. Gu et al., "Is your LLM secretly a world model of the Internet? Model-based planning for web agents," *Trans. Mach. Learn. Res.*, 2025.
- [30] J. Liu et al., "Mobile-Bench: An evaluation benchmark for LLM-based mobile agents," in Proc. Annu. Meeting Assoc. Computational Linguistics (ACL), 2025.
- [31] D. Koh, S. McAleer, D. Fried, and R. Salakhutdinov, "Tree search for language model agents," in Proc. Int. Conf. Learning Representations (ICLR), 2024.
- [32] H. Cui, Y. Du, Q. Yang, Y. Shao, and S. C. Liew, "LLMind: Orchestrating AI and IoT with LLM for complex task execution," *IEEE Communications Magazine*, 2025.
- [33] L. Cross, V. Xiang, A. Bhatia, D. L. K. Yamins, and N. Haber, "Hypothetical minds: Scaffolding theory of mind for multi-agent tasks with large language models," in Proc. Int. Conf. Learning Representations (ICLR), 2025.